

Sincronización de Procesos

Módulo 6

Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"

Sincronización de Procesos

- Fundamentos
- El Problema de la Sección Crítica
- La Solución de Peterson y otras
- Hardware de Sincronización
- Semáforos
- Problemas Clásicos de Sincronización
- Monitores
- Ejemplos de Sincronización

Fundamentos

- **Procesamiento concurrente** es la base de los SO modernos (multiprogramados)
- Los procesos cooperativos afectan o son afectados por otros procesos
- Los **procesos cooperativos** pueden compartir
 - un espacio de direcciones lógico (memoria compartida) ó
 - sólo los datos a través de archivos o paso de mensajes.
- El **acceso concurrente** a los datos compartidos puede generar incoherencias de los datos.
- Se debe proporcionar:
 - Mecanismos de **sincronización** y **comunicación** entre procesos.
 - Ejecución ordenada.

Fundamentos

Problema básico:

- Dos o más procesos quieren llevar a cabo una tarea **concurrentemente** y éstos pueden llegar a resultados incorrectos debido a que
 - no sabemos el orden de ejecución.
 - no sabemos cuando serán interrumpidos y su alternancia en la CPU.
- Ej: Productor-Consumidor
 - **Productor**: Genera información y la acumula en un buffer.
 - **Consumidor**: Accede al buffer y usa la información.

Problema del productor-consumidor



Fundamentos

- Sincronización:
 - Productor: No debe acceder al buffer si está lleno.
 - Consumidor: No debe acceder al buffer si está vacío.
 - Usan variables comunes.
- Solución al buffer limitado con memoria compartida
 - Al código del productor-consumidor se agrega una variable *contador*,
 - inicializada a 0 y
 - se incrementa cada vez que un nuevo elemento es agregado al buffer

Productor –Consumidor: buffer limitado

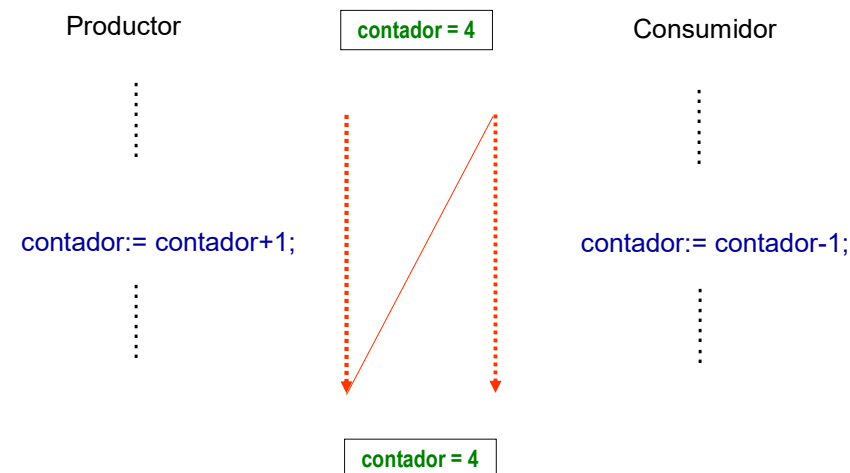
- **Datos compartidos**
`type item = ... ;`
`var buffer array [0..n-1] of item;`
`in, out: 0..n-1;`
`contador : 0..n;`
`in, out, contador := 0;`
- **Proceso Productor**
`repeat`
...
 produce un item en nextp
...
 while `contador = n` **do** *no-op*;
 `buffer[in] := nextp;`
 `in := in + 1 mod n;`
 `contador := contador + 1;`
`until false;`
- Proceso Consumidor**
`repeat`
 while `contador = 0` **do** *no-op*;
 `nextc := buffer[out];`
 `out := out + 1 mod n;`
 `contador := contador - 1;`
 ...
 consume el item en nextc
 ...
`until false;`

Las sentencias: `contador := contador + 1;`
 `contador := contador - 1;`
deben ser ejecutadas *atómicamente*.

Productor –Consumidor: buffer limitado

- Las siguientes variables residen en una zona de memoria compartida por los procesos productor y consumidor.
 - El buffer se implementa como un arreglo circular con dos punteros lógicos **in** y **out**
 - **in** apunta a la siguiente posición libre
 - **out** apunta a la 1ra posición ocupada
 - Si el buffer esta vacío $in == out$
 - Si el buffer está lleno $((in+1) \% buffer_size) == out$
 - **Contador** mantiene la cantidad de elementos en el buffer
- **nextp** es una variable local que produce el productor
- **nextc** es una variable local al consumidor en la que almacena el elemento que va a consumir

El Problema



Luego de una operación de Productor y otra de Consumidor ... contador permanece invariante

El Problema

Productor

Consumidor

$reg_a \leftarrow contador$
 $reg_a \leftarrow reg_a + 1$
 $contador \leftarrow reg_a$

$reg_b \leftarrow contador$
 $reg_b \leftarrow reg_b - 1$
 $contador \leftarrow reg_b$

$reg_a = 5$

$reg_b = 3$

$contador = 5$

JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

El Problema

P1

$a = b = 1$

P2

$a := a + 1;$
 $b := b + 1;$

$a := 2 * a;$
 $b := 2 * b;$

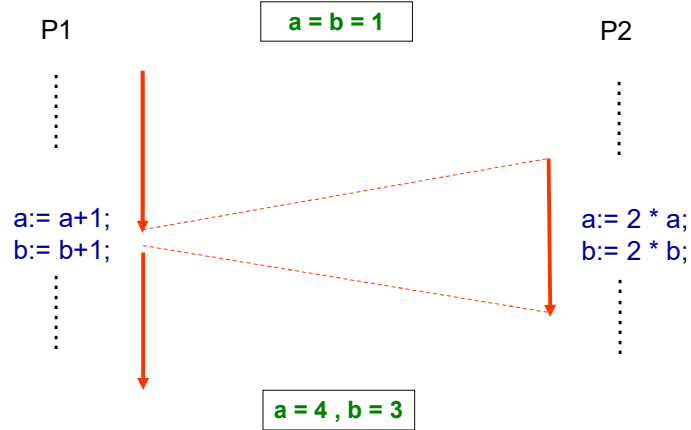
$a = b = 4$

Luego de la ejecución de P1 seguida de P2 los valores de a y b deben ser iguales entre sí

JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

El Problema

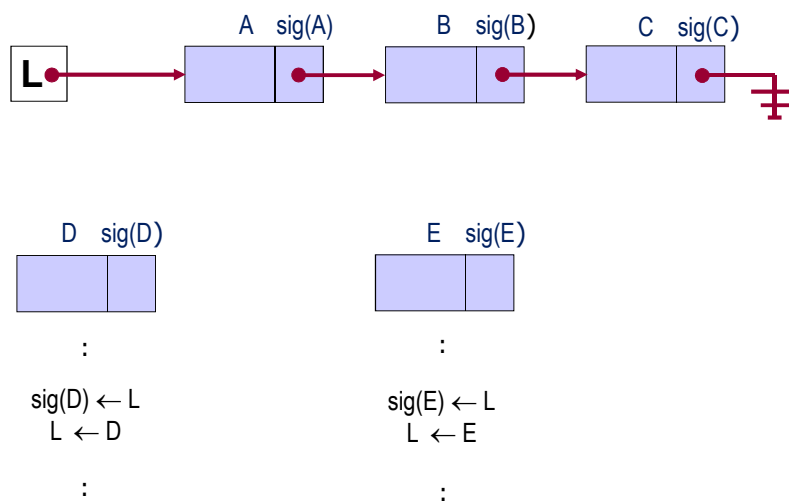


Luego de la ejecución de P1 y P2 los valores de a y b deberían ser iguales entre sí, pero no lo son.

JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

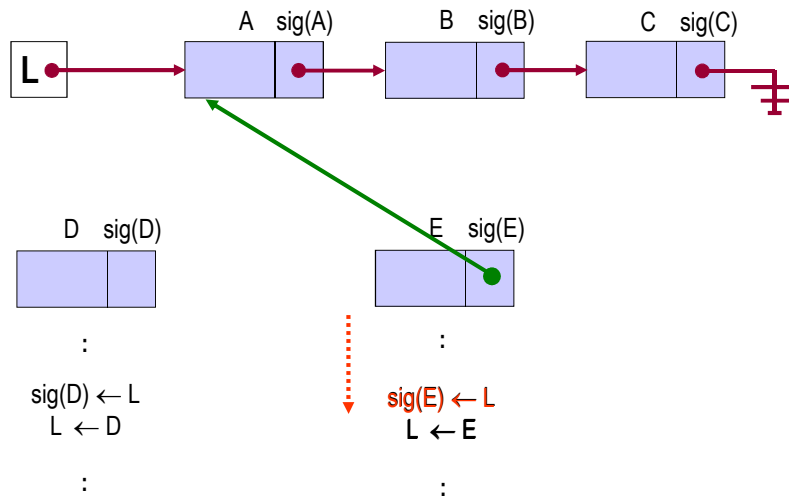
El Problema



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

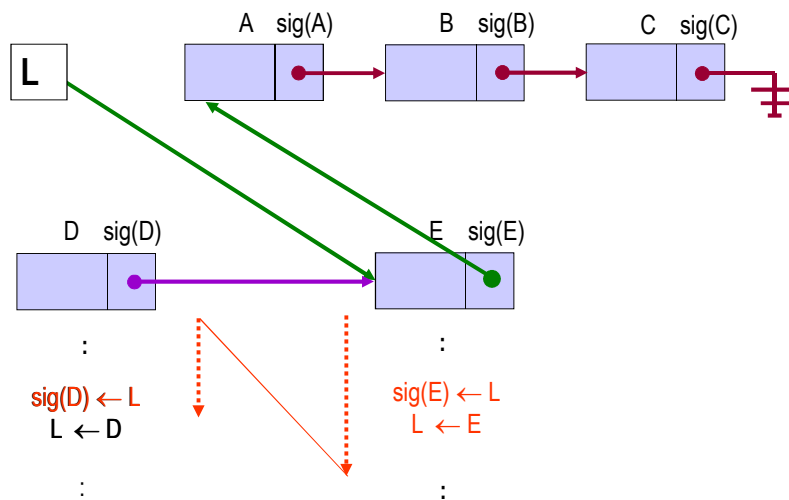
El Problema



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

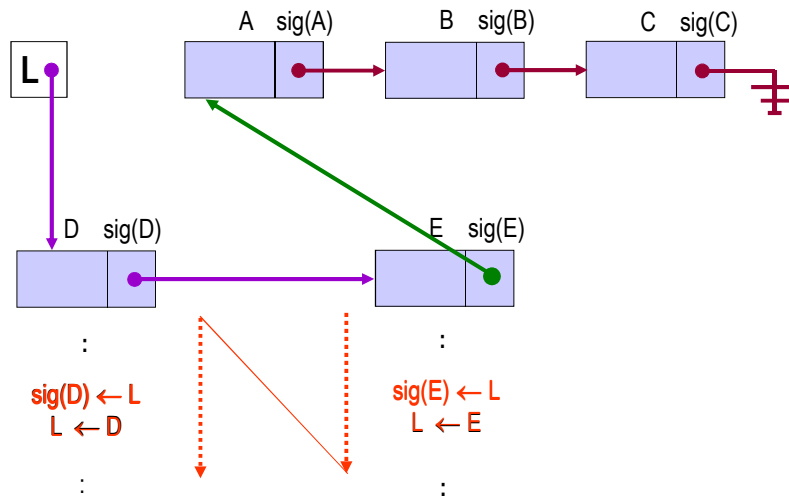
El Problema



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

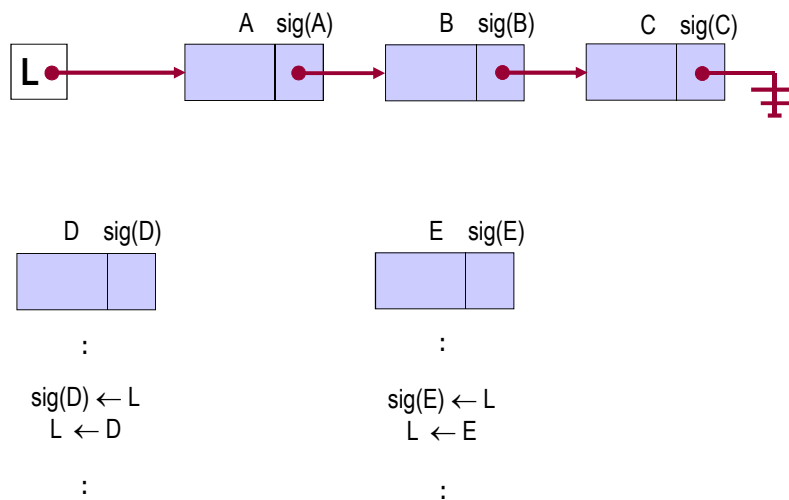
El Problema



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

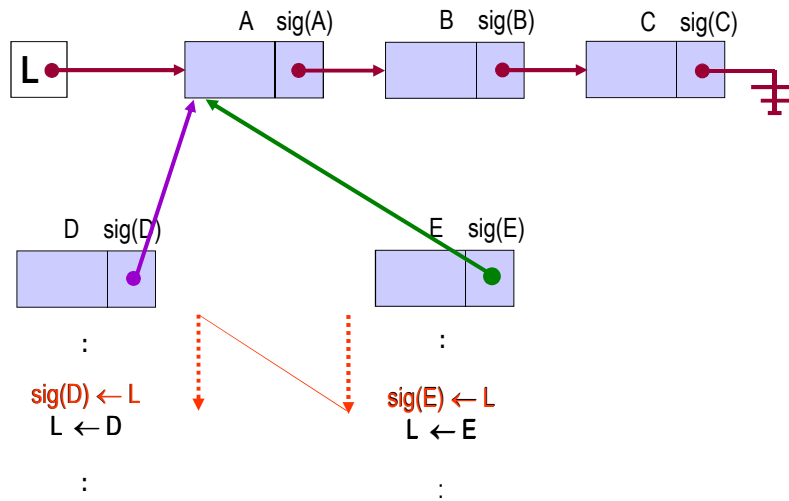
El Problema



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

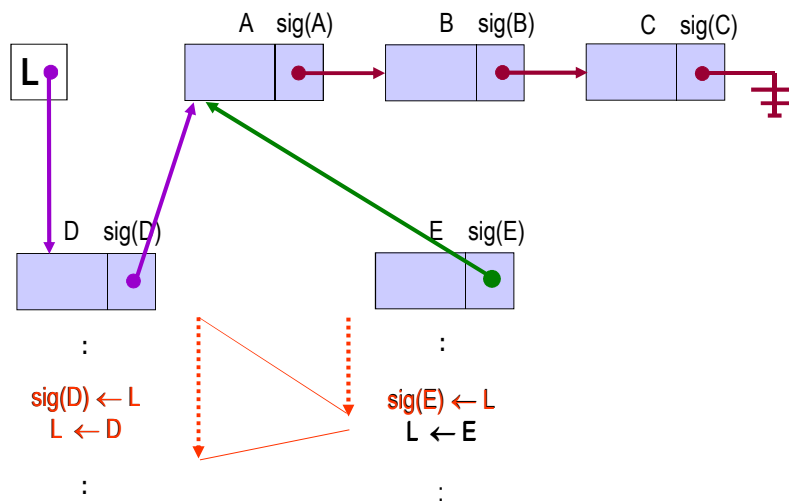
El Problema



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

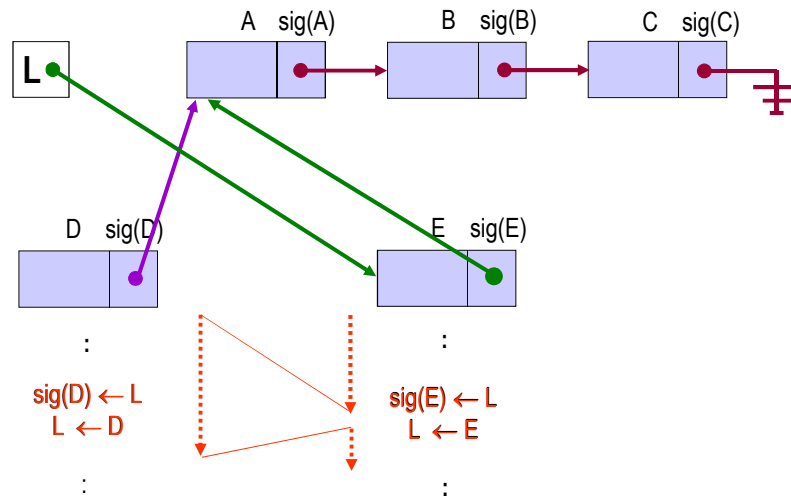
El Problema



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

El Problema



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

Condición de Carrera

- ◆ **Condición de carrera** –Es la situación donde varios procesos acceden y manejan datos compartidos concurrentemente. El valor final de los datos compartidos depende de que proceso termina último.
- ◆ Para prevenir las condiciones de carrera, los procesos concurrentes cooperativos deben ser **sincronizados**.

JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

Problema de la Sección Crítica

- Tenemos n procesos compitiendo para usar datos compartidos
- La **sección crítica (SC)**, de cada proceso, es el segmento de código, en el cual son accedidos los datos compartidos
- Problema – asegurar que cuando un proceso está ejecutando en su sección crítica, no se le permita a otro proceso ejecutar en su respectiva sección crítica.
- Las SC son **mutuamente excluyentes** en el tiempo

Solución al Problema de la Sección Crítica

1. **Exclusión Mutua.** Si el proceso P_i está ejecutando en su sección crítica, entonces **ningún otro proceso** puede estar ejecutando en sus secciones críticas.
 2. **Progreso.** Si ningún proceso está ejecutando en su sección crítica y existe algunos procesos que desean entrar en sus secciones críticas, entonces **la selección de estos procesos no puede ser pospuesta indefinidamente.**
 3. **Espera Limitada.** Debe existir un número limitado de veces que se le permite a otros procesos entrar en sus secciones críticas después que un proceso ha hecho un requerimiento para entrar en su sección crítica y antes que ese requerimiento sea completado.
- Se asume que cada proceso ejecuta a velocidad distinta de cero.
 - No se asume nada respecto a la velocidad relativa de los n procesos.

Resolución del Problema

- Estructura general del proceso P_i

repeat

protocolo de entrada

sección crítica (SC)

protocolo de salida

resto de sección

until falso

- Los procesos pueden compartir algunas variables comunes para sincronizar sus acciones.

Solución de Peterson

- Solo se aplica a 2 procesos, P_0 y P_1 ó P_i y P_j a la vez.

Proceso P_i

repeat

```
flag[i] := true;  
turn := j;  
while (flag[j] and turn=j)  
    do no-op;
```

sección crítica

```
flag[i] := false;
```

resto de sección

until falso;

Proceso P_j

repeat

```
flag[j] := true;  
turn := i;  
while (flag[i] and turn=i)  
    do no-op;
```

sección crítica

```
flag[j] := false;
```

resto de sección

until falso;

- Inicialmente $flag[0]=flag[1]= false$. (turn puede ser 0 ó 1)
- Alcanza los tres requerimientos; resuelve el problema de la sección crítica para dos procesos.

Algoritmo para N Procesos

(Algoritmo del "panadero")

Sección crítica para n procesos

- Antes de entrar en su sección crítica, el proceso recibe un número. El poseedor del número mas chico entra en su sección crítica.
- Si los procesos P_i y P_j reciben el mismo número, si $i < j$, entonces P_i es servido primero; sino lo es P_j .
- El esquema de numeración siempre genera números en orden incremental de enumeración;

p.e., 1,2,3,3,3,3,4,5...

Algoritmo para N Procesos(Cont.)

- **Notación** orden lexicográfico (ticket #, id proceso #)
 - $(a,b) < (c,d)$ si $a < c$ o si $a = c$ y $b < d$
 - $\max(a_0, \dots, a_{n-1})$ es un número k , tal que $k \geq a_i$ para $i = 0, \dots, n-1$

- Dato compartido

var choosing: array $[0..n-1]$ **of** *boolean*;

number: array $[0..n-1]$ **of** *integer*,

Las estructuras de datos son inicializadas a *false* y 0 respectivamente.

Choosing se pone en verdadero cuando el proceso i quiere obtener un nro para entrar a la sección crítica

Number es un vector que contiene el numero obtenido del proceso i

Algoritmo para N Procesos(Cont.)

repeat

```
choosing[i] := true;
number[i] := max(number[0], number[1], ..., number[n - 1]) + 1;
choosing[i] := false;
for j := 0 to n - 1
  do begin
    while choosing[j] do no-op;
    while number[j] ≠ 0
      and (number[j], j) < (number[i], i) do no-op;
  end;
```

sección crítica

```
number[i] := 0;
```

resto de sección

until false;

Sincronización por Hardware

- Muchos sistemas proveen soporte por hardware para resolver el problema de la sección crítica
- Monoprocesador – podría deshabilitarse las interrupciones
 - El código que está corriendo debe ejecutar *sin apropiación*.
 - En sistemas multiprocesador resulta demasiado ineficiente.
- Las computadoras modernas proveen instrucciones especiales que se ejecutan atómicamente
 - ▶ Atómico = no-interrumpible
 - (test-and-set)
 - (Swap)

Exclusión Mutua con Test-and-Set

- Verifica y modifica el contenido de una palabra atómicamente.

```
función Test-and-Set (var target: boolean): boolean;  
begin  
    Test-and-Set := target;  
    target := true;  
end;
```

- Dato compartido : var lock: boolean (inicialmente *false*)
- Proceso P_i

```
repeat  
    while Test-and-Set (lock) do no-op;  
    sección crítica  
    lock := false;  
    resto de sección  
until false;
```

Instrucción Swap

- Definición:

```
void Swap (boolean *a, boolean *b)  
{  
    boolean temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

- La variable booleana compartida *lock* es inicializada en FALSE;
Cada proceso tiene una variable local booleana *key*
- Solución usando Swap:

```
while (true) {  
    key = TRUE;  
    while ( key == TRUE)  
        Swap (&lock, &key );  
    // sección crítica  
    lock = FALSE;  
    // sección restante  
}
```

Semáforos

- Es una herramienta de sincronización.
- Semáforo **S** – variable entera
- Operaciones que modifican **S**: **wait()** y **signal()**
Originalmente llamadas **P()** y **V()** [término holandés **Proberen**, probar y **Verhoger**, incrementar]
- **S** puede ser accedido solo por dos operaciones indivisibles (atómicas):

```
wait (S): while S ≤ 0 do no-op;  
S := S - 1;
```

```
signal (S): S := S + 1;
```

Semáforo como Herramienta General de Sincronización

- Semáforo **Contador** o **cuenta** – valor entero puede variar en un rango sobre un dominio sin restricciones.
 - Usado para controlar el acceso a un determinado recurso con un nro de instancias.
 - Se inicializa con el número de recursos disponibles
 - El proceso que desee usar el recurso ejecuta **wait()** y decrementa el semáforo cuenta
 - Si libera el recurso, ejecuta **signal()** e incrementa **cuenta**.
 - Cuando todos los recursos están en uso, **cuenta** es 0.
- Semáforo **Binario** – valor entero, puede tomar sólo 0 ó 1;
 - simple de implementar
 - se los conoce como **mutex locks**

Semáforo como Herramienta General de Sincronización

- Provee exclusión mutua
 - Semáforo S ; // inicializado en 1
 - `wait (S);`
 - Sección Crítica
 - `signal (S);`

Ej de sincronización

- Ejecute B en P_j solo **después** que P_i ejecute A
- El semáforo $flag$ inicializado a 0



Implementación del Semáforo

- Podemos tener en la implementación del semáforo espera ocupada (*spinlock*):
 - Se espera q los semáforos se mantengan en un período corto
 - Si la sección crítica es raramente invocada, habrá poca espera ocupada
- Desventaja:
 - La espera ocupada o espera activa, desperdicia ciclos de CPU
 - las aplicaciones pueden pasar mucho tiempo en secciones críticas, entonces esta solución no es buena.

Implementación de Semáforo Sin Espera Ocupada

- Con cada semáforo hay asociada una cola de espera.
- Cada entrada en dicha cola tiene dos datos:
 - **valor** (de tipo entero)
 - puntero al próximo registro en la **lista**
- Se utilizan **dos operaciones**:
 - **block** – coloca al proceso en una cola de espera asociada al semáforo.
 - **wakeup** – remueve uno de los procesos en la cola de espera y lo ubica en la cola de listos.

Implementación (Cont.)

- Las operaciones del semáforo se definen como

```
wait(S): S.value := S.value - 1;  
    if S.value < 0  
    then begin  
        agregue este proceso a S.L;  
        block;  
    end;  
  
signal(S): S.value := S.value + 1;  
    if S.value ≤ 0  
    then begin  
        remueva un proceso P de S.L;  
        wakeup(P);  
    end;
```

Interbloqueo e Inanición

- **Interbloqueo** – ocurre cuando dos o más procesos están esperando indefinidamente por un evento que puede ser producido por solo uno de los procesos que esperan.
- Sean S y Q dos semáforos inicializados a 1

P_0	P_1
<code>wait(S);</code>	<code>wait(Q);</code>
<code>wait(Q);</code>	<code>wait(S);</code>
<code>:</code>	<code>:</code>
<code>signal(S);</code>	<code>signal(Q);</code>
<code>signal(Q);</code>	<code>signal(S);</code>

- **Inanición** – puede ocurrir un **bloqueo indefinido**. Un proceso no puede ser removido nunca de la cola del semáforo en el que fue suspendido.

Problemas Clásicos de Sincronización

- Problema del Buffer Limitado
- Problema de Lectores y Escritores
- Problema de los Filósofos Cenando

Problema del Buffer Limitado

■ Datos compartidos

```
type item = ...  
var buffer = ...  
full, empty, mutex: semaphore;  
nextp, nextc: item;  
full := 0; empty := n; mutex := 1;
```

■ Proceso Productor

```
repeat  
  ...  
  produce un ítem en nextp  
  ...  
  wait(empty);  
  wait(mutex);  
  ...  
  agregue nextp al buffer  
  ...  
  signal(mutex);  
  signal(full);  
until false;
```

■ Proceso Consumidor

```
repeat  
  wait(full)  
  wait(mutex);  
  ...  
  remueve un ítem de buffer a nextc  
  ...  
  signal(mutex);  
  signal(empty);  
  ...  
  consume el ítem en nextc  
  ...  
until false;
```

Problema de los Lectores-Escritores

■ Pensemos en una Base de Datos compartida entre un número de procesos concurrentes

- Lectores – solo leen el conjunto de datos, no realizan actualizaciones
- Escritores – pueden leer y/o escribir

■ Problema – permitir a múltiples lectores leer al mismo tiempo si hay algún escritor. Un solo escritor puede acceder a los datos compartidos al mismo tiempo.

■ Datos compartidos

- Conjunto de datos
- Semáforo **mutex** inicializado a 1.
- Semáforo **wrt** inicializado a 1.
- Entero **readcount** inicializado a 0.

Problema de los Lectores-Escritores

■ Datos compartidos

```
var mutex, wrt: semaphore (=1);  
readcount : integer (=0);
```

■ Proceso Escritor

```
wait(wrt);  
...  
se realiza la escritura  
...  
signal(wrt);
```

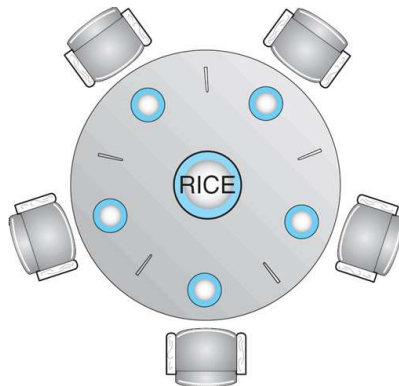
■ Proceso Lector

```
wait(mutex);  
  readcount := readcount + 1;  
  if readcount = 1 then wait(wrt);  
signal(mutex);  
...  
se realiza la lectura  
...  
wait(mutex);  
  readcount := readcount - 1;  
  if readcount = 0 then signal(wrt);  
signal(mutex);
```

Problema de los Filósofos Cenando

■ Datos compartidos

- ◆ Fuente con arroz (conjunto de datos) y 5 palillos (recurso)
- ◆ **var palillo: array [0..4] of semáforo; (=1 inicialmente)**



Filósofo i:

```
repeat  
  wait(palillo [i])  
  wait(palillo [i+1 mod 5])  
  ...  
  comer  
  ...  
  signal(palillo [i]);  
  signal(palillo [i+1 mod 5]);  
  ...  
  pensar  
  ...  
until false;
```

Problemas con Semáforos

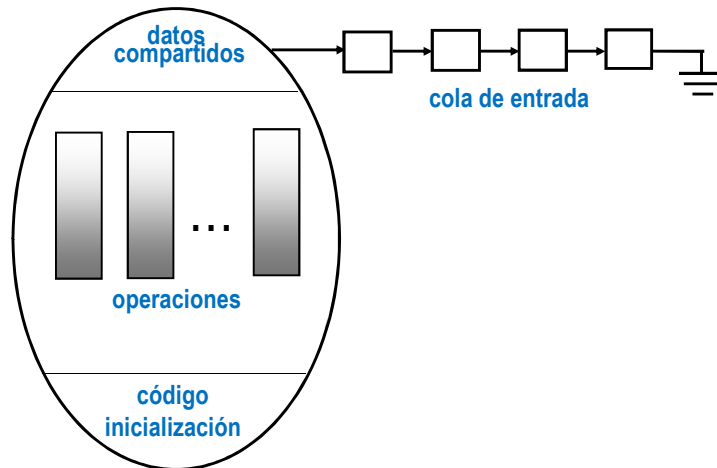
- Puede ocurrir un uso incorrecto de las operaciones sobre semáforos:
 - signal (mutex) wait (mutex)
 - wait (mutex) ... wait (mutex)
 - Omitir el wait (mutex) o el signal (mutex) o ambos

Monitores

- Es un constructor de sincronización de alto nivel que permite compartir en forma segura un tipo de dato abstracto entre procesos concurrentes.

```
type monitor-name = monitor
  declaraciones de variables
  procedure entry P1 (...);
    begin ... end;
  procedure entry P2 (...);
    begin ... end;
    ⋮
  procedure entry Pn (...);
    begin...end;
begin
  código de inicialización
end
```

Vista esquemática de un Monitor



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

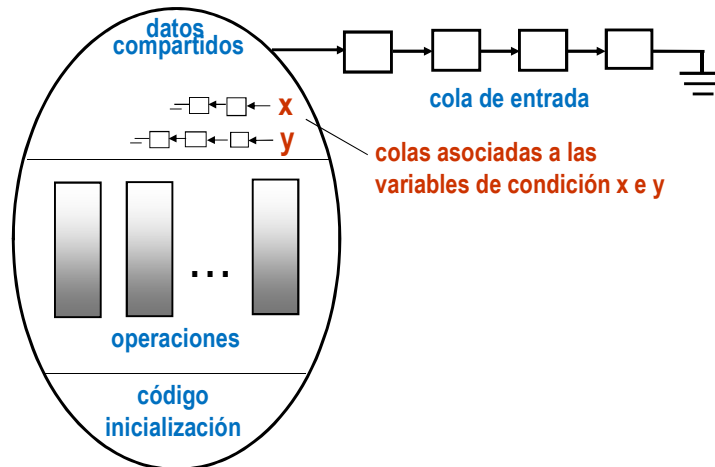
Variables de Condición

- La estructura monitor asegura que sólo un proceso esté activo a la vez dentro del monitor
- El monitor asegura exclusión mutua.
- Para lograr la **sincronización** necesitamos mecanismos adicionales.
 - Un programador puede definir una o mas variables de tipo condición para la sincronización.
- condición x, y ;
- Dos operaciones sobre una variable de condición
 - $x.wait$ – el proceso que invoca esa operación es suspendido
 - $x.signal$ – reinicia uno de los procesos (si hay alguno) que invocó $x.wait$

JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

Monitor con Variables de Condición



JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

Solución de los Filósofos Cenando

type filósofos-cenando = **monitor**

var estado: array [0..4] of (pensando, hambre, comiendo);

var self : array [0..4] of condition;

procedure entry pickup (i: 0..4);

begin

estado[i] := hambre;

test (i);

if estado[i] ≠ comiendo
 then self[i].wait;

end;

procedure entry putdown (i: 0..4);

begin

estado[i] := pensando;

test (i+4 mod 5);

test (i+1 mod 5);

end;

procedure test(k: 0..4);

begin

if estado[k+4 mod 5] ≠ comiendo

and estado[k] = hambre

and estado[k+1 mod 5] ≠ comiendo

then begin

 estado[k] := comiendo;

 self[k].signal;

end;

end;

begin

for i := 0 to 4 **do**

 estado[i] := pensando;

end.

JRA © - LRS 2025

Sistemas Operativos – Sincronización de Procesos

Solución de los Filósofos Cenando

- La distribución de los palillos se controla con el monitor **dp**.
- Cada filósofo invoca las operaciones `pickup()` y `putdown()` en la siguiente secuencia:

`dp.pickup (i)`

...

Comer

...

`dp.putdown (i)`

Ejemplos de Sincronización

- Solaris
- Windows XP
- Linux
- Pthreads

Sincronización en SOLARIS

- Implementa una variedad de **locks** para soportar multitasking, multithreading (incluyendo threads en tiempo real), y multiprocesamiento.
- Usa **mutex adaptativos** para mayor eficiencia, protegiendo el acceso a los datos críticos, para segmentos de código cortos. (con espera activa –**Spinlock**)
- Usa **variables de condición** y semáforos **locks lectores-escritores** cuando necesitan acceder a los datos, dentro de grandes secciones de código.
- Usa cola de bloqueos (**turnstile**) para ordenar la lista de threads esperando para adquirir un mutex adaptativo o un lock lectores-escritores

Sincronización Windows XP

- Windows tiene un kernel multihilo para dar soporte a aplicaciones de tiempo real y multiprocesador.
- Usa **spinlocks** (bloqueo con espera activa) en sistemas multiprocesador
- En sistemas monoprocesador, deshabilita temporalmente las interrupciones para proteger el acceso a recursos globales.
- También provee **dispatcher objects** para sincronizar los hilos fuera del kernel, empleando diferentes mecanismos: mutex, semáforos, eventos o señales y temporizadores.
 - Un **evento** actúa como una variable de condición y notifica a un hilo en espera, que a sucedido la condición deseada.
 - Se emplea un **temporizador** para notificar a un hilo (o mas) que transcurrió un determinado período de tiempo.

Sincronización en Linux

- Linux:
 - Deshabilita las interrupciones para implementar secciones críticas cortas
- provee:
 - Semáforos (lectores - escritores)
 - spin locks

Sincronización en Pthreads

- APIs Pthreads son independientes de los SOs, no forman parte de ningún kernel.
- Proveen:
 - Locks mutex
 - Variables de condición
 - Locks lector-escritor
- Extensiones no portables incluyen:
 - spin locks

Fin

Módulo 6

Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"